Running RawTFNet-16 on QEMU vs Raspberry Pi 4

Notes on emulator feasibility and expected performance for the TrustZone/OP-TEE proof-of-concept

---

Can it run?

Yes. The RawTFNet-16 model (Best_RawTFNet_16.pth) has already been exported to ONNX and compiled with Apache TVM on the QEMU/OP-TEE setup, with output verified to match the PyTorch/ONNX reference. Getting inference running end-to-end is mainly a matter of wiring the compiled model into a Trusted Application (TA) / Client Application (CA) pair, or calling it from a normal-world app that invokes the secure-world TA.

Will it be slower than a real Pi 4?

Yes, noticeably. Two separate factors compound:

- **Not a hardware-accurate emulation.** The QEMU *virt* platform used for OP-TEE is a generic ARMv8-A virtual machine — it does not model the Pi 4's actual Cortex-A72 cores, cache hierarchy, or memory bandwidth.
- **No hardware acceleration for the CPU emulation itself.** Since the OP-TEE build runs inside a VirtualBox Ubuntu VM (on an x86_64 host), QEMU performs full software instruction translation (TCG) from ARM64 to x86_64 — there is no KVM acceleration for ARM-on-x86. CPU-bound workloads such as the matrix multiplications in neural network inference are exactly the kind of operation TCG handles worst.

On top of this, the stack is VirtualBox → QEMU → OP-TEE, adding a second layer of virtualization overhead before instruction translation even begins.

Expected magnitude

TCG-translated CPU-bound workloads are commonly 5–20× slower than native execution, and sometimes more depending on how well the operation mix vectorizes. A real Pi 4 (Cortex-A72 @ 1.5 GHz) running natively should beat the QEMU/TCG setup by a wide margin for RawTFNet-16 inference — likely in the range of one to two orders of magnitude, depending on how the model's operations map to TCG's translation efficiency.

Implication for the thesis

QEMU/OP-TEE is best treated as a **functional** proxy for TrustZone development — proving that confidential inference works, validating the TA/CA architecture and secure-world isolation — rather than a **performance** proxy. If real-time latency numbers are needed to support the "live in-call detection" claim, the recommended path is to benchmark on actual Pi 4 hardware, or at minimum explicitly note in the thesis that QEMU timings are not representative and provide a rough scaling estimate.

Suggested next step

Add timing instrumentation inside the TA to capture clean inference latency numbers from the QEMU run, which can then be compared against Pi 4 figures later if hardware becomes available.